Project 2 Title

Sigurd Riseth
Department of Physics
University of Oslo
Oslo, Norway
sigurri@uio.no

***Abstract*—abstract.**

***Index Terms*—classification, regression, neural networks,**

I. Introduction

TODO: introduction.

The remainder of this report is structured as follows. Section II presents the ... Section III describes the data, the preprocessing, the code and how it was verified. Section IV presents and discusses the results in the order ..., and section V summarizes our findings and outlines future work. Appendix A declares our use of large language models. The code and results behind every figure and table, are available in our publicly hosted repository [1].

II. Methods

This section presents the models, ... We assume the reader has familiarity with basic linear algebra and statistics.

A. Logistic regression

Logistic regression is a natural choice as a baseline. It is a simple model in which everything can be computed, its cost function is convex, and it is exactly a neural network with no hidden layers [2].

1) Error measures: We evaluate a prediction $\tilde{\boldsymbol{y}}$ of targets $\boldsymbol{y}$ by

B. Neural networks

1) The artificial neuron:

2) Activation functions: Our study will revolve around the following three activation functions:

a) Sigmoid: The logistic or sigmoid function is given by

$$\sigma(z) = \frac{1}{1+e^{-z}},$$

b) ReLU: The rectified linear unit (ReLU),

$$\mathrm{ReLU}(z) = \max(0, z),$$

is faster to compute

The leaky ReLU is defined as

$$\mathrm{LReLU}(z) = \begin{cases} z, & \text{if } z > 0, \\ \alpha z, & \text{if } z \leq 0, \end{cases} \qquad \alpha \approx 0.01.$$

III. Implementation

We implemented all methods ourselves in Python using NumPy [3], with the full source code, tests, and experiment scripts available in the project repository [1]. Below, we cover the data, preprocessing, code structure, and how we verified the implementation.

A. Data and preprocessing

For the regression task we use Runge's function [4].

For the classification task we use the classic MNIST dataset [5], a collection of 70 000 handwritten digits and their corresponding labels (i.e., 0–9).

B. Code structure

We reuse the structure and the gradient machinery of the previous project [6]. The code is structured in a package `fys_stk4155_p2`, containing the directories `nn`, `optimization`; which respectively hold the code for the models and gradient descent.

Thin constructor of the FFNN for logistic regression, which returns a single layer with softmax activation and cross entropy loss. This allows us to use this as a baseline model when evaluating the later neural networks.

C. Verification

IV. Results and Discussion

TODO: results and discussion.

V. Conclusion

TODO: conclusion.

References

[1] S. Riseth, *FYS-STK4155 Project 2*, version v0.1.0, GitHub, Nov. 2026. [Online]. Available: https://github.com/sigurdriseth/FYS-STK4155-P2

[2] M. Hjorth-Jensen, *Machine Learning and the Physical Sciences: From Discriminative Methods to Generative Methods and Reinforcement Learning*. Springer, Oct. 9, 2026. [Online]. Available: https://github.com/EducationalMaterialUiO / MachineLearningUiO / blob / main/doc/BookML/Lecturebook.pdf

[3] C. R. Harris et al., "Array programming with NumPy," *Nature*, **585**, no. 7825, pp. 357–362, Sep. 17, 2020, ISSN: 0028-0836, 1476-4687. DOI: 10.1038/s41586-020-2649-2 Accessed: Aug. 23, 2026. [Online]. Available: https://www.nature.com/articles/s41586-020-2649-2

[4] C. Runge, "Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten," *Zeitschrift für Mathematik und Physik*, **46**, pp. 224–243, 1901.

[5] Y. LeCun, C. Cortes, and C. J. Burges, "MNIST handwritten digit database," *ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist*, **2**, 2010.

[6] S. Riseth, *FYS-STK4155 Project 1*, version v0.1.0, GitHub, Oct. 2026. [Online]. Available: https://github.com/sigurdriseth/FYS-STK4155-P1

APPENDIX

A. Use of AI/LLM tools

Large language models were used throughout this project, following the course guidelines[1], which also define the contribution levels below. All code was tested, all derivations were checked, and every claim in the report was read, understood and verified by the author.

1) Tools used:

- TODO: tools, models, and dates.

TODO: summary of how each tool was used.

2) Text writing and editing: Table I gives the contribution level for each part of the report.

TABLE I
LLM CONTRIBUTION TO TEXT WRITING AND EDITING (LEVELS 0–3).

Section	Level	Notes
Abstract	TODO	
Introduction	TODO	
Methods	TODO	
Implementation	TODO	
Results and discussion	TODO	
Conclusion	TODO	
Appendices	TODO	

3) Code generation and assistance: Table II gives the level for each source file (levels 0–4). Functions with level 2 or above carry an `LLM-assisted` tag in their docstring.

4) Declared and undeclared use: TODO: one example of a use that must be declared, and one example of a use that need not be declared (e.g. using an LLM as a tutor and then writing about it in your own words).

B. Additional results

TODO: additional results.

[1] https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/LLM_Usage_Declaration_Guidelines.md

TABLE II
LLM CONTRIBUTION TO THE CODE (LEVELS 0–4).

File	Level	Description
TODO	TODO	